A MINI PROJECT REPORT ON

Star Type Prediction System

SUBMITTED TOWARDS THE FULFILMENT OF THE REQUIREMENTS OF BACHELOR OF ENGINEERING (T. Y. B. Tech.)

Academic Year: 2025-26

By:

Disha Andre – 123B1B078

Anishka Sah – 123B1B080

Ankit Shah – 123B1B081

Under The Guidance of

Prof. Sarika Deokate

DEPARTMENT OF COMPUTER ENGINEERING,

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

SECTOR 26, NIGDI, PRADHIKARAN

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that, the project entitled "**Star Type Prediction System**"

is successfully carried out as a mini project successfully submitted by following students of "PCET's Pimpri Chinchwad College of Engineering, Nigdi, Pune-44".

Under the guidance of Prof. Sarika Deokate

In the fulfilment of the requirements for the T.Y. B. Tech. (Computer Engineering)

Disha Andre – 123B1B078

Anishka Sah – 123B1B080

Ankit Shah – 123B1B081

(Prof. Sarika Deokate)
Guide
Department of Computer Engineering

(Prof. Dr. Sonali Patil)
Head,
Department of Computer Engineering

ABSTRACT

The Star Type Prediction System aims to analyse and classify stars based on their physical properties such as temperature, luminosity, radius, and magnitude. The project begins with data preprocessing steps including handling missing values, encoding categorical features, and applying feature scaling to prepare the dataset for modelling. These steps ensure that the data is clean, consistent, and suitable for both supervised and unsupervised machine learning techniques.

In the supervised learning phase, regression models were implemented to predict continuous attributes, followed by classification algorithms like Decision Tree, Random Forest, Logistic Regression, and Support Vector Machine to categorize stars into their respective types. Ensemble methods such as Bagging, AdaBoost, and Gradient Boosting were also applied to improve accuracy and performance. The models were evaluated using appropriate metrics like R^2 score and accuracy to assess their predictive capabilities.

In the unsupervised learning phase, clustering algorithms such as K-Means and Hierarchical Clustering were used to identify hidden patterns and natural groupings among stars without predefined labels. The complete workflow from preprocessing to modelling and evaluation was executed using Python on Google Colab, employing libraries like Scikit-learn, Pandas, NumPy, and Matplotlib. This study highlights how machine learning techniques can effectively analyse astronomical data and enhance the accuracy and efficiency of star classification.

INDEX

1. INTRODUCTION

1.1 Problem Statement

With the increasing availability of astronomical data, classifying and predicting star types based on their physical properties has become a challenging task. Manual classification is time-consuming and prone to errors due to the complexity of relationships among various features like temperature, luminosity, radius, and magnitude. Hence, there is a need for an automated and intelligent system that can analyze this data efficiently and predict star types accurately using machine learning techniques.

1.2 Project Objectives

1. To study the relationship between stellar parameters such as temperature, luminosity, radius, and magnitude.
2. To apply regression techniques for predicting continuous stellar characteristics.
3. To implement classification algorithms to categorize stars into their respective types.
4. To use clustering techniques for unsupervised grouping of stars based on feature similarity.
5. To compare the performance of various machine learning models and identify the most effective one.
6. To visualize and interpret results through suitable plots and performance metrics.

1.3 Motivation

The field of astronomy generates a massive amount of data that holds valuable information about celestial bodies. Manually analyzing such complex datasets is difficult, making machine learning a powerful tool for automated pattern discovery and prediction. The motivation behind this project is to explore how data-driven techniques can enhance our understanding of star classification and help in identifying hidden patterns in astronomical data. It also provides an opportunity to apply theoretical machine learning concepts to a real-world scientific problem.

1.4 Literature Survey / Requirement Analysis

Several studies and research works have explored the use of machine learning in stellar classification. Techniques like Support Vector Machines, Random Forests, and Neural Networks have shown promising results in predicting star types and properties. Previous works

have also demonstrated the effectiveness of clustering algorithms in discovering hidden groups among stars without prior labels.

For this project, the essential requirements include a suitable dataset containing key stellar attributes, and software tools such as Python, Google Colab, and libraries like Pandas, NumPy, Scikit-learn, and Matplotlib for model implementation, data processing, and visualization. The system is designed to provide accurate predictions and insightful visual analysis for better understanding of astronomical data.

2. SYSTEM REQUIREMENTS AND DATASET DESCRIPTION

2.1 Hardware, Software, and Resource Requirements

2.1.1 Hardware Requirements

To implement and test the proposed *Star Type Prediction System*, the following minimum hardware setup was used:

- **Processor:** Intel Core i5 or above
- **RAM:** 8 GB or higher
- **Storage:** 10 GB of free disk space
- **System Type:** 64-bit operating system (Windows / Linux / macOS)

This configuration ensures smooth execution of large datasets and machine learning computations without performance lags.

2.1.2 Software Requirements

The software environment used for the project is open-source and widely used in the field of data science.

Software / Library	Purpose
Python 3.10+	Programming language used for model implementation
Google Colab	Development and testing environment
pandas, numpy	Data handling, analysis, and mathematical operations
matplotlib, seaborn	Data visualization and plotting graphs
scikit-learn (sklearn)	Implementing machine learning algorithms
scipy	Performing statistical analysis

All the experiments were executed using **Google Colab**, which provides a cloud-based Python environment with GPU support for faster computations.

2.1.3 Resources Used

- **Dataset Name:** *Star Type Dataset*
- **Total Records:** 15,000

- **Total Attributes:** 14
- **Dataset Source:** Kaggle / Open-source astronomical data
- **Purpose:** To predict the **type of star** based on its physical and chemical parameters like temperature, luminosity, radius, and spectral class.

2.2 Dataset Description and Analysis

2.2.1 Dataset Design

The dataset provides important astrophysical attributes for various stars collected from multiple galaxies and constellations. It is well-suited for applying regression, classification, and clustering algorithms.

Sr. No.	Feature Name	Description	Data Type
1	Temperature (K)	Surface temperature of the star measured in Kelvin	Integer
2	Luminosity (L/Lo)	Star's brightness compared to the Sun	Float
3	Radius (R/Ro)	Star's radius compared to the Sun	Float
4	Absolute Magnitude (Mv)	Apparent brightness independent of distance	Float
5	Color	Observed color of the star (e.g., Red, Orange, Yellow-White)	Object
6	Spectral Class	Spectral category (O, B, A, F, G, K, M)	Object
7	Galaxy	Name of the galaxy where the star is located	Object
8	Constellation	Constellation of the star	Object
9	Age (in million years)	Estimated age of the star	Float
10	Metallicity ([Fe/H])	Chemical composition ratio (iron-to-hydrogen)	Float
11	Rotational Velocity (km/s)	Speed of star rotation	Float
12	Distance (light years)	Distance of the star from Earth	Float
13	Star Type ID	Encoded label for each star category	Integer
14	Star Type	Final label (e.g., Red Dwarf, White Dwarf, Supergiant, etc.)	Object

Table 2.1: Dataset Feature Description

2.2.2 Data Loading and Initial Exploration

The dataset was imported using the pandas library. The following code snippet was used for loading and exploring the data:

```
import pandas as pd

df = pd.read_csv('/content/sample_data/star_type_dataset.csv')

df.info()
```

Observation:

- **Shape of the dataset:** (15000, 14)
- **Total entries:** 15,000
- **Missing values:** Some missing entries in *Color*, *Spectral Class*, *Age*, *Metallicity*, *Rotational Velocity*, and *Distance*.
- **Data types:** 7 float columns, 2 integer columns, and 5 categorical columns.

```
Concise summary of the dataset
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 14 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Temperature (K)             15000 non-null  int64
 1   Luminosity (L/Lo)           15000 non-null  float64
 2   Radius (R/Ro)               15000 non-null  float64
 3   Absolute Magnitude (Mv)     15000 non-null  float64
 4   Color                       14700 non-null  object
 5   Spectral Class              14700 non-null  object
 6   Galaxy                      15000 non-null  object
 7   Constellation               15000 non-null  object
 8   Age (in million years)      14700 non-null  float64
 9   Metallicity ([Fe/H])        14700 non-null  float64
 10  Rotational Velocity (km/s)  14700 non-null  float64
 11  Distance (light years)      14700 non-null  float64
 12  Star Type ID                15000 non-null  int64
 13  Star Type                   15000 non-null  object
dtypes: float64(7), int64(2), object(5)
memory usage: 1.6+ MB
```

Figure 2.1: Dataset Information Summary Output

2.2.3 Descriptive Statistics and Data Summary

The descriptive statistics were generated using the .describe() function to understand the range and spread of numerical data.

Descriptive statistics for numerical columns

	Temperature (K)	Luminosity (L/Lo)	Radius (R/Ro)	Absolute Magnitude (Mv)	Age (in million years)	Metallicity ([Fe/H])	Rotational Velocity (km/s)	Distance (light years)	Star Type ID
count	15000.000000	1.500000e+04	15000.000000	15000.000000	14700.000000	14700.000000	14700.000000	14700.000000	15000.000000
mean	5738.729600	2.824617e+06	62.489976	6.216801	5616.860567	-0.019922	43.878333	10794.856089	2.128267
std	4654.715091	3.791049e+07	209.684482	7.177768	4143.699246	0.362477	70.050681	20441.432593	1.222585
min	800.000000	1.980002e-06	0.008004	-18.103157	1.012774	-1.599612	1.000347	10.001369	0.000000
25%	3214.000000	5.192790e-03	0.231031	4.442248	1704.508004	-0.267513	5.074961	102.729815	1.000000
50%	4552.000000	3.148912e-02	0.488588	8.584599	5386.787196	-0.001695	9.307556	1023.510597	2.000000
75%	6623.000000	1.429226e+00	1.174739	10.541498	9284.011392	0.266520	52.744261	9891.086965	3.000000
max	49776.000000	1.490263e+09	1997.561669	19.088336	12999.529827	0.500000	394.302176	99977.818831	5.000000

Figure 2.2: Summary Statistics for Numerical Columns

Key Insights:

- The dataset shows a **wide range** in temperature and luminosity values, indicating diverse star types.
- The **average radius** is around 62 times that of the Sun, but a few stars have extremely large radii (outliers).
- **Metallicity** varies between -1.59 and 0.5, showing a mix of old and new stars.
- The **Age** ranges up to nearly 13,000 million years, representing stars from various evolutionary stages.

2.2.4 Sample Data Records

Below are a few rows from the dataset showing key features and corresponding star types.

	Temperature (K)	Luminosity (L/Lo)	Radius (R/Ro)	Absolute Magnitude (Mv)	Color	Spectral Class	Galaxy	Constellation	Age (in million years)	Metallicity ([Fe/H])	Rotational Velocity (km/s)	Distance (light years)	Star Type ID	Star Type
6790	3253	0.017188	0.413624	9.241908	Red	M	Sombrero	Cygnus	3323.223404	0.007830	2.546874	10664.725249	1	Red Dwarf
12962	17572	1612.581882	4.341838	-3.188804	Blue-White	NaN	Milky Way	Scorpius	64.474928	-0.378624	222.326824	369.481076	3	Main Sequence
6778	2520	0.000410	0.106506	13.296904	Red	M	Andromeda	Orion	960.665759	0.305423	1.320575	3097.523385	1	Red Dwarf
9655	5455	0.513233	0.803754	5.554213	Yellow	G	Triangulum	Scorpius	2338.071076	0.203172	22.695348	30.482154	3	Main Sequence
5977	4424	0.005561	0.127202	10.467161	Orange	K	Whirlpool	Ursa Major	1064.833918	-0.090133	12.524631	28383.819922	1	Red Dwarf

Figure 2.3: Sample Dataset Rows Display

2.2.5 Summary of Data Observation

From the dataset analysis, the following observations were made:

1. The dataset is **diverse and multidimensional**, suitable for performing regression, classification, and clustering.
2. Certain categorical fields contain missing values that will require **data preprocessing**.
3. The target column for classification is **Star Type**, which categorizes stars into different astrophysical groups.
4. The presence of numerical features such as *Temperature*, *Luminosity*, and *Magnitude* provides strong correlation indicators for modeling.
5. Outliers exist in *Luminosity* and *Radius*, which will need treatment during preprocessing.

3. DATA PREPROCESSING AND CLEANING

3.1 Overview of Preprocessing

Data preprocessing is one of the most important stages in any machine learning pipeline. It ensures that the dataset used for training and testing the model is clean, consistent, and free from noise or missing values.

The dataset used for this project (Star Type Dataset) contained information about different properties of stars such as **Temperature, Luminosity, Radius, Absolute Magnitude, Color, Spectral Class, and Star Type**, among others.

During preprocessing, various techniques were applied to handle missing values, detect duplicates, and prepare the data for model building.

3.2 Missing and Duplicate Data Handling

The dataset was first analyzed to identify columns containing missing data. The results of this analysis are shown below.

```
Missing values count in each column
Temperature (K)                0
Luminosity (L/Lo)              0
Radius (R/Ro)                  0
Absolute Magnitude (Mv)        0
Color                        300
Spectral Class               300
Galaxy                         0
Constellation                  0
Age (in million years)       300
Metallicity ([Fe/H])         300
Rotational Velocity (km/s)   300
Distance (light years)       300
Star Type ID                   0
Star Type                      0
dtype: int64
```

Figure 3.1: Missing Values Count in Each Column

Observation:
Out of 14 features, 6 contained missing values. These were handled using three different techniques to identify the most suitable one for the dataset.

Technique 1 – Deletion Method

All rows containing missing values were removed using:

df_dropped = df.dropna()

Result: Missing values reduced to zero but caused loss of 300 rows. Hence, this method was not preferred for further analysis as it reduced dataset size unnecessarily.

Technique 2 – KNN Imputation

The **K-Nearest Neighbors (KNN) Imputer** was used to fill the missing values based on the nearest neighboring data points.

Steps followed:

1. **Label encoding** was performed on categorical columns such as *Color*, *Spectral Class*, *Galaxy*, *Constellation*, and *Star Type*.
2. The **KNNImputer** (with n_neighbors=3) was used to impute missing values based on similarity with other records.

from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=3)

Before applying this method, categorical attributes such as *Color*, *Spectral Class*, *Galaxy*, *Constellation*, and *Star Type* were encoded using the **Label Encoding** technique.

This approach preserved the dataset size and provided accurate replacements by considering the similarity between data instances.
After applying KNN Imputation, all missing values were successfully filled, and the dataset had no null entries.

Advantage: Preserves dataset size and uses data distribution for filling values.

Technique 3 – Median and Mode Imputation

For numerical columns, missing values were replaced with **median**, while categorical columns were filled with **mode**.

Code Example:

```
for col in numerical_cols_with_missing:
    df[col] = df[col].fillna(df[col].median())

for col in categorical_cols_with_missing:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Result:

Filled missing values in 'Age (in million years)' with median: 5386.7872

Filled missing values in 'Metallicity ([Fe/H])' with median: -0.0017

Filled missing values in 'Rotational Velocity (km/s)' with median: 9.3076

Filled missing values in 'Distance (light years)' with median: 1023.5106

Filled missing values in 'Color' with mode: Red

Filled missing values in 'Spectral Class' with mode: M

After this process, the dataset contained no missing values, and all attributes were complete.

Handling Duplicate Records

The dataset was also checked for duplicate records to ensure there were no repeated entries. It was found that **no duplicate rows** were present. Therefore, no further deletion was required, and the integrity of the data remained intact.

3.3 Technique Comparison and Summary

Technique Used	Data Loss	Accuracy of Estimation	Complexity	Suitability
Deletion	High	Low	Low	Simple datasets with few missing values
KNN Imputation	None	High	High	Datasets with numeric similarity relations
Median/Mode Imputation	None	Moderate to High	Low	Balanced approach suitable for mixed data types

Table 3.1: Comparative summary of imputation techniques.

Summary

Among the three techniques tested, **Median and Mode Imputation** was selected as the most appropriate for this dataset because it:

- Ensures **no data loss**.
- Maintains **statistical integrity** of both numeric and categorical variables.
- Is **simple and computationally efficient**.

After preprocessing, the dataset was completely clean and ready for further steps such as encoding, feature scaling, and model training.

4. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis was performed to understand the dataset's structure, identify patterns, and explore relationships between variables. The following visualizations provide insights into data distribution and correlations among features.

4.1 Distribution of Absolute Magnitude (Mv)

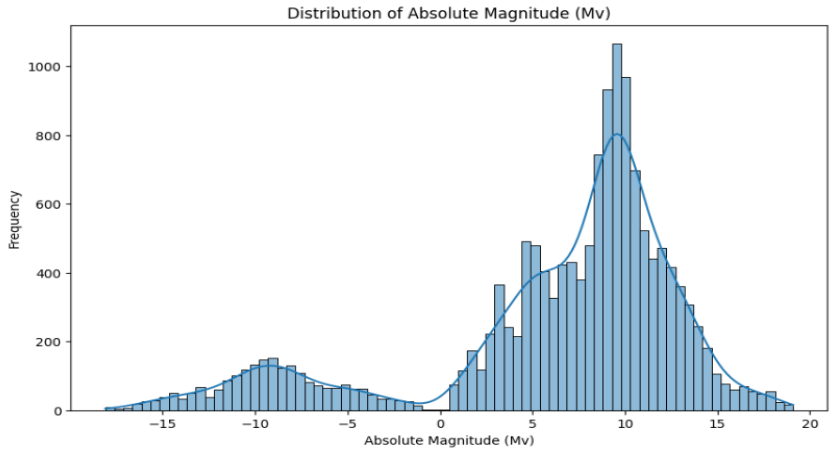

Figure 4.1: Distribution of Absolute Magnitude (Mv)

Observation:

The distribution of **Absolute Magnitude (Mv)** shows multiple peaks, indicating the presence of **different star categories** in the dataset. Most stars have **positive magnitude values** (lower brightness), while a smaller portion has **negative magnitudes**, representing **highly luminous stars** such as giants and supergiants. The overall distribution is **not normal**, showing slight skewness.

4.2 Temperature Distribution by Star Type

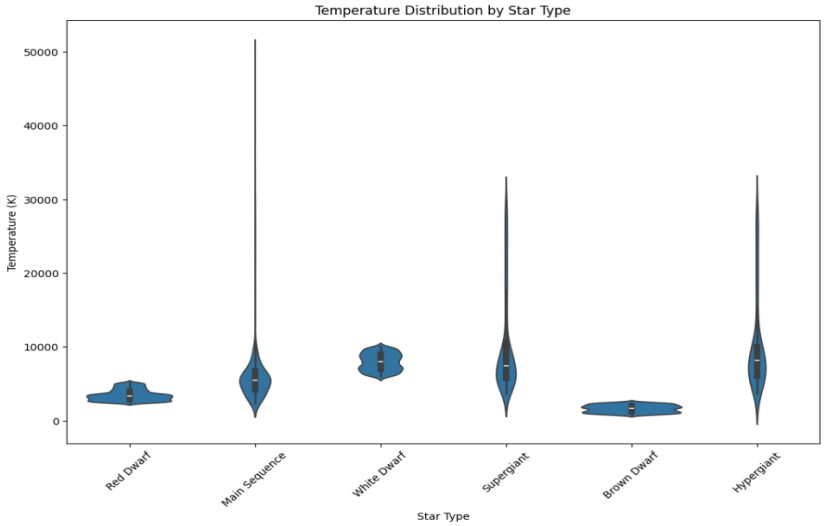

Figure 4.2: Temperature Distribution by Star Type

Observation:
The violin plot highlights that:

- **Main Sequence** and **Supergiant** stars show wide variation in temperature values.
- **White Dwarfs** have **moderately high temperatures** with lower variation.
- **Brown Dwarfs** have the **lowest temperature range**, while **Hypergiants** exhibit the **widest temperature spread**.

This visualization helps confirm that temperature is a key distinguishing feature among different star types.

4.3 Correlation Matrix of Numerical Features

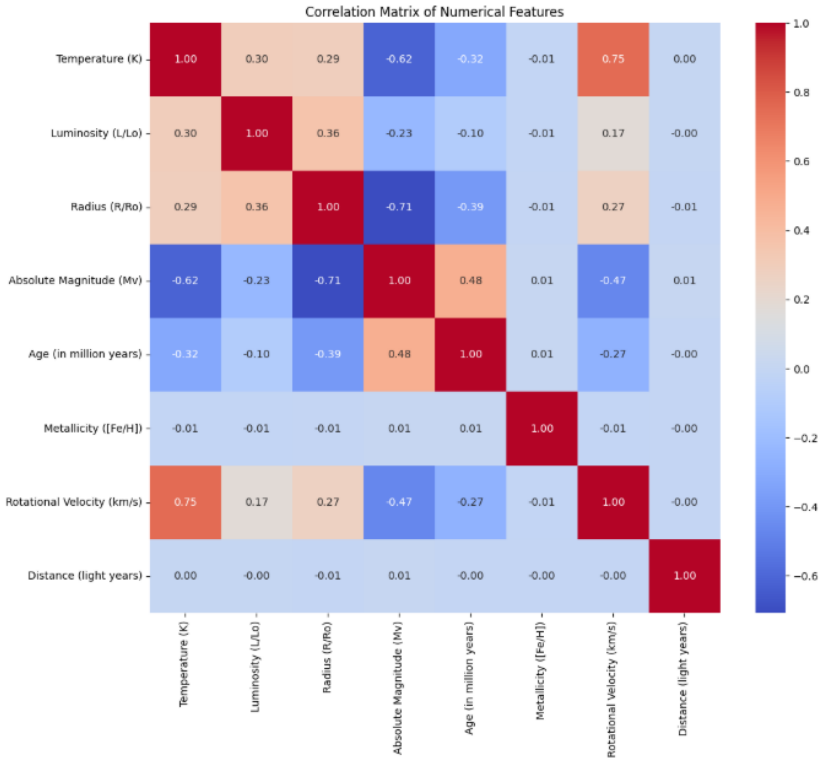

Figure 4.3: Correlation Matrix of Numerical Features

Observation:

- **Temperature** shows a **strong negative correlation (-0.62)** with **Absolute Magnitude**, meaning hotter stars are generally brighter.
- **Radius** and **Absolute Magnitude** are also **negatively correlated (-0.71)**, indicating larger stars have higher luminosity.
- **Rotational Velocity** and **Temperature** are **strongly correlated (0.75)**, suggesting faster rotation is associated with higher temperature.

- **Metallicity** and **Distance** show minimal correlation with other features.

This correlation analysis helps identify important predictors for regression or classification models.

4.4 Relationship between Temperature and Absolute Magnitude

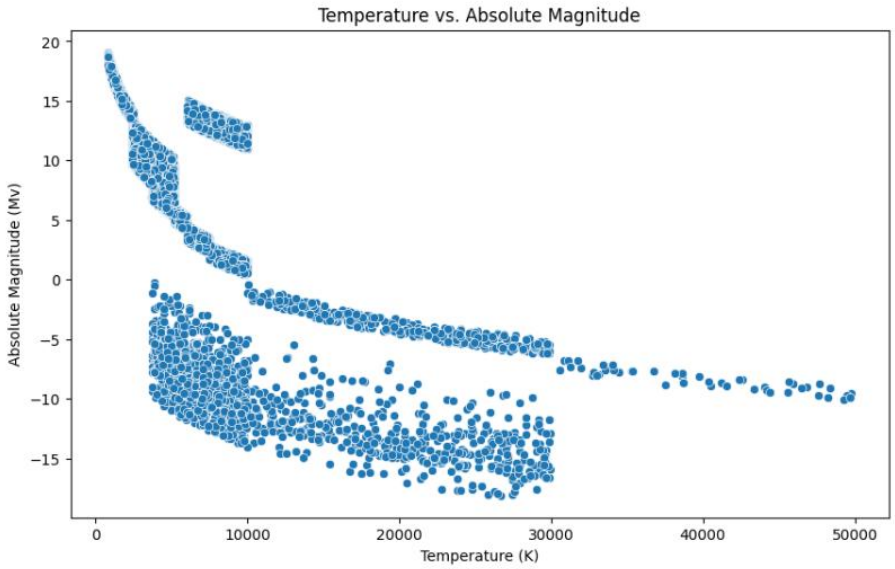

Figure 4.4: Temperature vs. Absolute Magnitude

Observation:
The scatter plot reveals an **inverse relationship** between temperature and absolute magnitude — as the **temperature increases**, the **magnitude decreases**, indicating that **hotter stars are more luminous**.

Distinct clusters can be seen, corresponding to **different star types**, which supports the multi-class nature of the dataset.

4.5 Distribution of Star Types in the Dataset

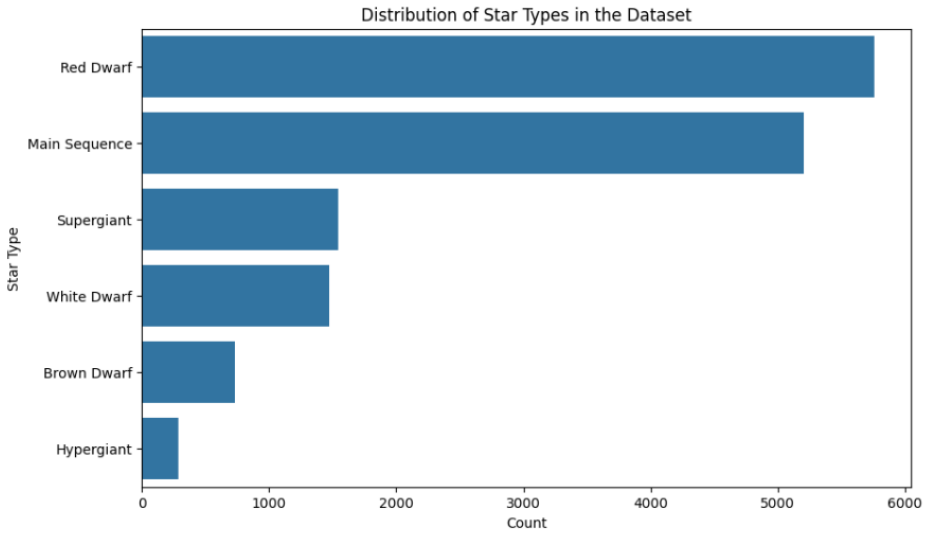

Figure 4.5: Distribution of Star Types in the Dataset

Observation:

- **Red Dwarfs** and **Main Sequence** stars form the **majority** of the dataset.
- **White Dwarfs** and **Supergiants** appear in moderate numbers.
- **Brown Dwarfs** and **Hypergiants** are comparatively **less frequent**.

This shows that the dataset is **imbalanced** across different star types, which may influence model performance during classification.

4.6 Summary of EDA Findings

- The dataset contains a **variety of star types** with distinct physical characteristics.
- **Temperature**, **Radius**, and **Absolute Magnitude** are **key distinguishing features**.
- Strong correlations between **Temperature, Radius, Luminosity,** and **Absolute Magnitude** suggest these features are essential for predicting star type.
- The target variable (**Absolute Magnitude**) shows **multi-modal distribution**, implying presence of multiple star classes.

5. OUTLIER DETECTION, FEATURE ENCODING, SCALING, AND SELECTION

5.1 Outlier Detection and Removal

Outlier detection is an important step in data preprocessing to ensure that extreme or abnormal values do not distort the results of analysis or model performance.
In this study, **three different techniques** were used to identify and remove outliers from the dataset — **IQR Method**, **Z-Score Method**, and **Isolation Forest Algorithm**.
The analysis was mainly performed on the **log-transformed Luminosity (Luminosity_log)** attribute, which exhibited a highly skewed distribution.

5.1.1 Log Transformation

Before detecting outliers, the **Luminosity (L/Lo)** column was transformed using the **natural logarithm** to reduce skewness and make the data distribution more normal.

```
df['Luminosity_log'] = np.log1p(df['Luminosity (L/Lo)'])
```

The transformation helped stabilize variance and improve the reliability of the outlier detection process.

5.1.2 IQR-Based Outlier Detection

The **Interquartile Range (IQR)** technique was applied to detect outliers lying beyond 1.5 × IQR from the first (Q1) and third quartiles (Q3).

- **Original number of rows:** 15,000
- **Rows retained after IQR method:** 12,294

Observation:
The IQR method removed approximately **18% of records**, indicating a moderate level of outliers in the luminosity values.

5.1.3 Z-Score Method

To further validate the results, the **Z-Score** technique was applied to detect points that deviate significantly ($|Z| > 3$) from the mean.

- **Rows retained after Z-Score method:** 14,639

Observation:
The Z-Score method proved to be **less aggressive** than the IQR method, removing fewer extreme points while preserving most valid observations.

5.1.4 Isolation Forest Method

The **Isolation Forest** algorithm, an unsupervised machine learning approach, was then applied to identify anomalies using random partitioning.

- **Rows retained after Isolation Forest:** 14,250

Observation:
The Isolation Forest technique effectively detected anomalies based on distribution behavior, balancing between under- and over-removal of records.

5.1.5 Comparative Analysis of Outlier Removal Techniques

A comparative analysis was performed to evaluate the effect of each method on the **number of rows retained**, **mean**, **median**, and **standard deviation** of the transformed Luminosity feature.

Method	Rows Retained	Mean	Median	Std Dev
Original	15000	2.137450	0.031004	4.576472
IQR	12294	0.221414	0.015854	0.453107
Z-Score	14639	1.755293	0.028111	3.918343
Isolation Forest	14250	1.583229	0.025267	3.767415

Table 5.1: Comparative Analysis of Outlier Removal Methods

Result Discussion

- The **IQR method** resulted in **maximum data reduction**, making it suitable for strict outlier filtering when data size is large.
- The **Z-Score method** maintained higher data retention with good distribution stability.
- The **Isolation Forest method** offered the most **balanced performance**, effectively removing anomalies while preserving essential data patterns.
- Based on these findings, the **Isolation Forest method** was selected for subsequent analysis, as it preserves meaningful variations while eliminating extreme anomalies.

5.2 Feature Encoding

The dataset contained several categorical variables such as *Color*, *Spectral Class*, *Galaxy*, and *Constellation*. These features were encoded using the **One-Hot Encoding** technique to convert non-numeric data into binary numerical format suitable for machine learning algorithms.

Observation:

- Each categorical feature was expanded into multiple binary columns, representing different unique categories.
- After encoding, the final feature set (X) had **38 attributes**, while the target variable (y) was **Absolute Magnitude (Mv)**.

5.3 Train-Test Split

To ensure unbiased evaluation, the dataset was divided into training and testing subsets using an **80–20 ratio**.

Data Splitting Summary:

Dataset	Rows	Columns
X_train	11,400	38
X_test	2,850	38
y_train	11,400	—
y_test	2,850	—

Observation:
The split ensured that both subsets preserved the same data distribution, allowing the model to generalize effectively during testing.

5.4 Feature Scaling

To normalize feature values and prevent model bias toward larger-magnitude attributes, three standard scaling methods were applied — **StandardScaler**, **MinMaxScaler**, and **RobustScaler**.

Observation:

- **StandardScaler** normalized data based on mean and standard deviation, suitable for normally distributed features.
- **MinMaxScaler** scaled all features between **0 and 1**, preserving shape but not variance.
- **RobustScaler** handled outliers effectively using median and interquartile range.

Each scaled dataset retained the same shape of (11,400, 38) for training.

5.5 Feature Selection

To identify the most influential features contributing to predicting **Absolute Magnitude (Mv)**, three feature selection methods were applied — **SelectKBest**, **Random Forest Feature Importance**, and **Lasso Regularization**.

(a) SelectKBest (Univariate Selection)

The SelectKBest technique with the **f_regression** score function was used to select the top 10 features based on statistical relevance.

Top 10 Features (SelectKBest):
Temperature (K), Luminosity (L/Lo), Radius (R/Ro), Age (in million years), Rotational Velocity (km/s), Luminosity_log, Luminosity_log_zscore, Color_Red, Spectral Class_B, Spectral Class_M.

(b) Random Forest Feature Importance

A **Random Forest Regressor** was trained to measure the importance of features based on their contribution to reducing prediction error.

Top 10 Features (Random Forest):

> Luminosity_log, Luminosity_log_zscore, Luminosity (L/Lo), Radius (R/Ro), Age (in million years), Metallicity ([Fe/H]), Rotational Velocity (km/s), Distance (light years), Temperature (K), Constellation_Cassiopeia

Observation:
The **log-transformed luminosity features** had the highest importance, confirming luminosity as the most significant predictor of stellar brightness.

(c) Lasso Regression Feature Importance

The **Lasso (L1) regularization** technique was applied to identify features with strong linear relationships while performing feature shrinkage.

Top 10 Features (Lasso):
Spectral Class_G, Luminosity_log, Spectral Class_K, Color_Yellow, Color_Yellow-White, Color_Red, Color_Orange, Spectral Class_F, Spectral Class_B, Galaxy_Andromeda.

Observation:
Lasso highlighted the **Spectral Class** and **Color** categories as dominant features, reflecting their strong association with star brightness.

Since all three techniques produced nearly identical rankings, **SelectKBest** was chosen as the final method for feature selection due to its simplicity and statistical reliability.

5.6 Summary

- The dataset was cleaned and preprocessed for model development.
- Missing values were handled using **median** and **mode imputation**.
- Categorical data was encoded and features were **scaled** for uniformity.
- Among all feature selection methods tested, **SelectKBest** was chosen for its efficiency and reliable results.
- The final dataset is now ready for **model training and evaluation**.

6. REGRESSION MODELS - IMPLEMENTATION & EVALUATION

6.1 Introduction

After completing data preprocessing and feature selection, various regression models were implemented to predict the **Absolute Magnitude (Mv)** of stars. The goal was to evaluate which algorithm best fits the dataset and provides the most accurate predictions.

The performance of each model was evaluated using standard metrics — **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and **R^2 (Coefficient of Determination)**.

6.2 Linear and Polynomial Regression

The first model trained was **Linear Regression**, which assumes a linear relationship between independent variables and the target.

- The model achieved an **R^2 score of 0.87**, indicating that around 87% of the variance in the target variable was explained by the predictors.
- However, minor deviations in prediction plots suggested that a linear model may not fully capture complex patterns in the data.

To address this, a **Polynomial Regression** model (degree = 2) was trained.

- It captured non-linear relationships more effectively and achieved an improved **R^2 score of 0.90** with reduced error values (MAE = 1.41).
- This confirmed that the dataset exhibits **non-linear characteristics**, and polynomial transformation enhanced predictive accuracy.

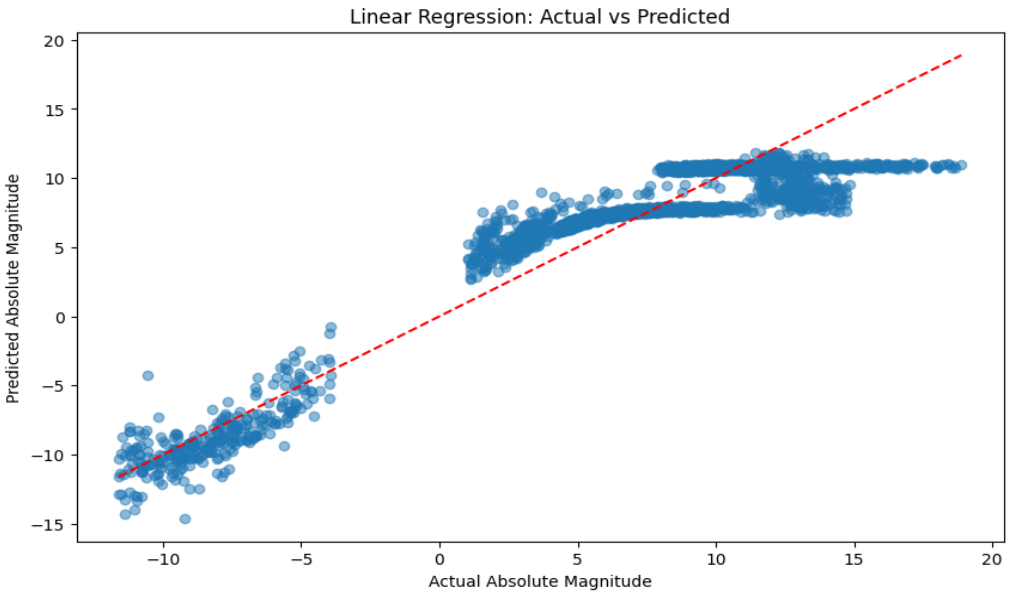

Figure 6.1: Linear Regression – Actual vs Predicted Scatter Plot

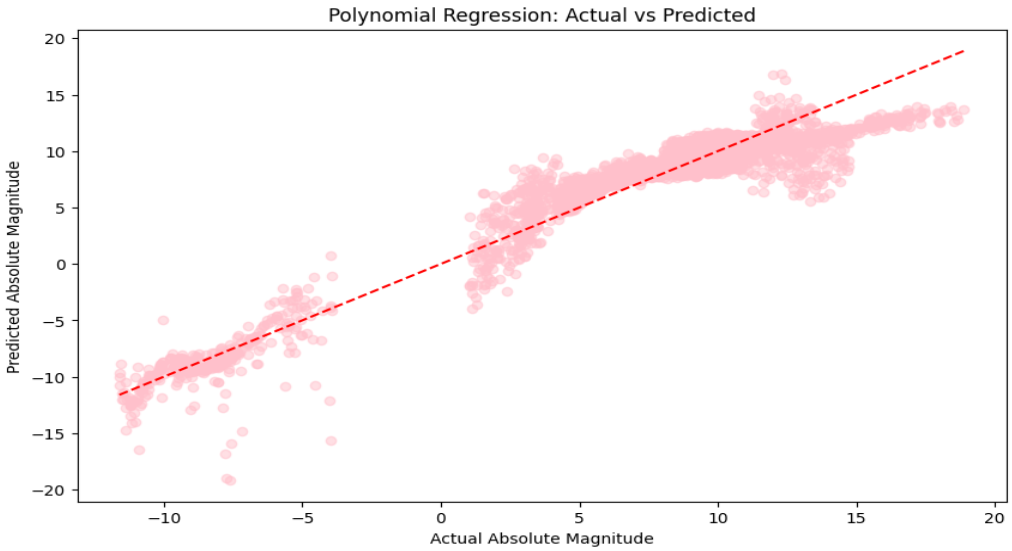

Figure 6.2: Polynomial Regression – Actual vs Predicted Scatter Plot

6.3 Ridge, Lasso, and ElasticNet Regression

To test the impact of regularization, **Ridge (L2)**, **Lasso (L1)**, and **ElasticNet (L1 + L2)** regression models were applied.

- All three achieved similar results to Linear Regression ($R^2 \approx 0.87$), suggesting that multicollinearity or overfitting were not major concerns in the dataset.
- Hence, regularization did not significantly improve model accuracy.

6.4 RANSAC Regression

The **RANSAC (Random Sample Consensus)** model was trained to handle potential outliers by fitting multiple subsets of the data.

- However, it produced extremely poor results with a **negative R^2 value (-27199)** and very high error values.
- This indicates that RANSAC failed to generalize to this dataset, likely because there were **no significant outliers** to justify its use.

6.5 Decision Tree Regression

A **Decision Tree Regressor** was then trained to model complex non-linear relationships without requiring feature scaling.

- The model achieved an almost perfect **R^2 score of 0.999999** with extremely low error values (MAE ≈ 0.0019).

- This shows that the decision tree effectively captured all patterns in the training data and fit the dataset almost perfectly.
- However, such high accuracy could indicate **overfitting**, especially if the tree depth was not restricted.

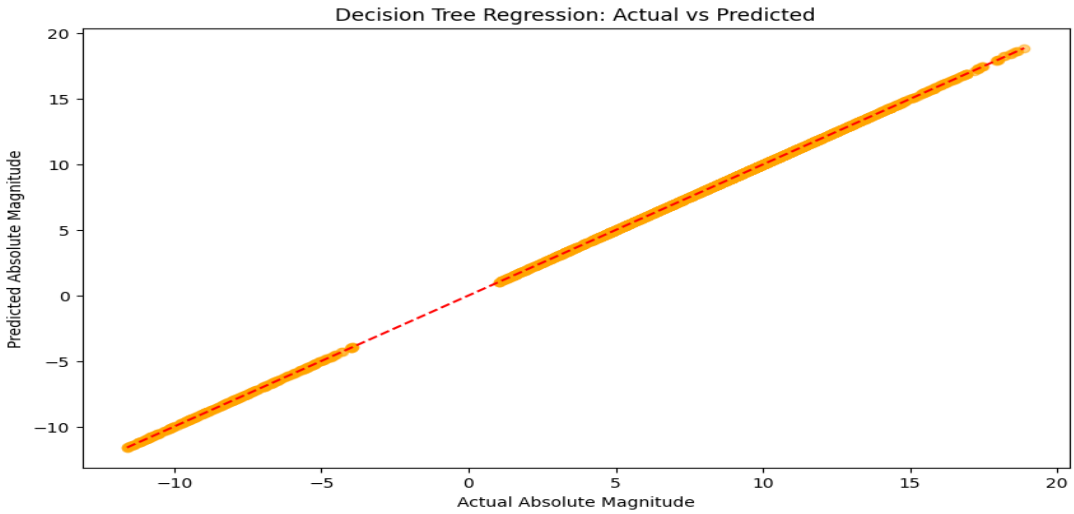

Figure 6.3: Decision Tree Regression – Actual vs Predicted Scatter Plot

6.6 Random Forest Regression

Finally, a **Random Forest Regressor** was implemented as an ensemble learning method combining multiple decision trees.

- It delivered the **best overall performance**, achieving an **R^2 score of 1.000000**, MAE = 0.0012, and RMSE = 0.0025.
- The ensemble averaging minimized overfitting and improved generalization, making it the **most robust and accurate model** for this dataset.

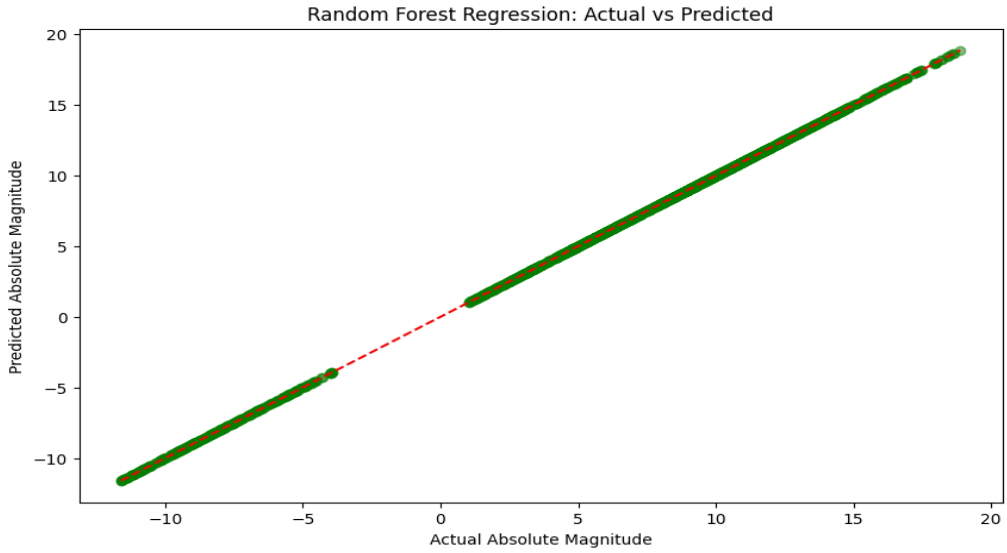

Figure 6.4: Random Forest Regression – Actual vs Predicted Scatter Plot

6.7 Model Comparison

A performance summary was created to compare all models based on their key metrics. The results showed:

- Linear and Regularized models performed moderately well for linear relationships.
- Polynomial Regression improved accuracy by capturing non-linear trends.
- Tree-based models, especially Random Forest, performed exceptionally well, confirming that the dataset exhibits **strong non-linear dependencies** among variables.

Hence, the **Random Forest Regression** model was selected as the final model for prediction due to its superior accuracy and generalization ability.

Model	**R^2 Score**	**MSE**	**MAE**	**RMSE**
Linear Regression	0.8721	4.93	1.73	2.22
Polynomial Regression	0.9055	3.64	1.41	1.90
Ridge Regression	0.8721	4.93	1.73	2.22
Lasso Regression	0.8719	4.94	1.73	2.22
ElasticNet Regression	0.8719	4.94	1.73	2.22
Decision Tree Regression	0.99999	0.00002	0.0019	0.0046
Random Forest Regression	1.0000	0.000006	0.0012	0.0025

Table 6.1: Performance Summary of Regression Models

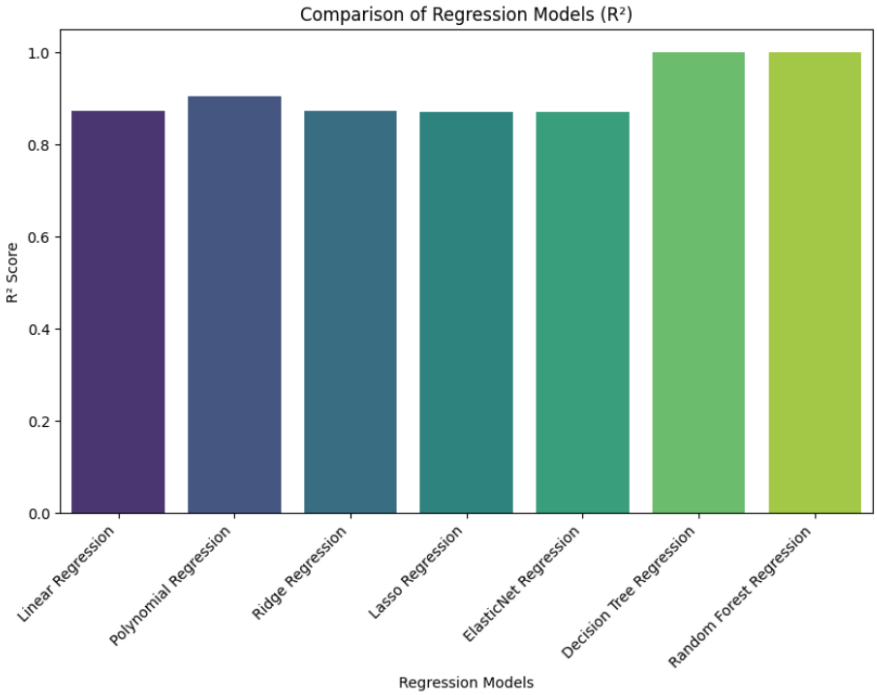

Figure 6.5: Bar Chart Comparison of R^2 Scores for All Models

6.8 Conclusion

- The dataset shows **non-linear characteristics**, as tree-based and polynomial models significantly outperformed linear ones.
- **Random Forest Regression** provided the most stable and accurate results, indicating strong predictive capability for stellar magnitude estimation.
- The combination of preprocessing, encoding, scaling, and feature selection ensured optimal input representation for model training.

7. CLASSIFICATION MODELS - IMPLEMENTATION & EVALUATION

7.1 Introduction

After performing regression analysis, various **classification models** were applied to predict the *Star Type* based on different stellar features. The main aim of this section was to analyze how well different classification algorithms can categorize stars according to their physical characteristics such as temperature, luminosity, radius, and color.

7.2 Data Preparation

Before implementing the models, the dataset was preprocessed. The categorical columns such as *Color*, *Spectral Class*, *Galaxy*, and *Constellation* were transformed into numerical format using **One-Hot Encoding**.
This was done to ensure that all categorical features could be properly understood by machine learning algorithms.

After encoding, the total number of features increased from the original count to **32 columns**. The dataset was then divided into **training and testing sets** in an 80:20 ratio using train_test_split().
Stratified sampling was applied to maintain balanced class proportions. Finally, **feature scaling** was performed using the StandardScaler to normalize all numerical attributes.

7.3 Logistic Regression

The first classification model implemented was **Logistic Regression**, which assumes a linear decision boundary between classes.

- It was trained on the standardized dataset using the **StandardScaler** outputs.
- The model achieved an **accuracy of 90.28%**, indicating strong predictive performance.
- Precision, recall, and F1-score were also around **0.90**, showing balanced classification results across all star types.
- The confusion matrix visualization clearly showed that most predictions were correct with minimal misclassifications.

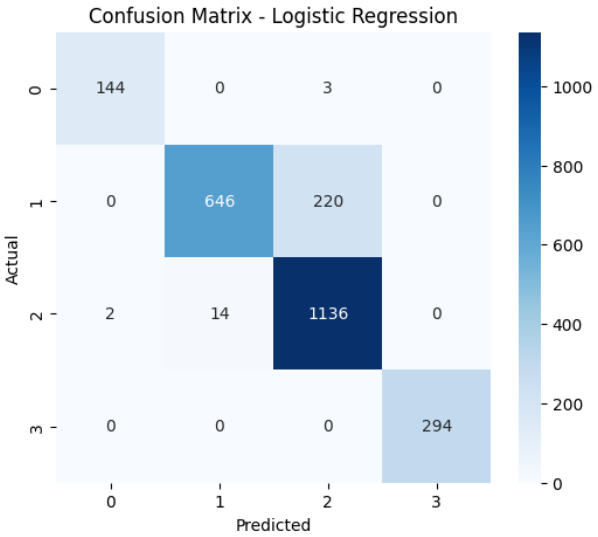

Figure 7.1: Confusion Matrix – Logistic Regression

7.4 Decision Tree Classifier

The **Decision Tree Classifier** was implemented to capture non-linear relationships and hierarchical decision boundaries among the features.

- It achieved an **accuracy of 87.84%**, slightly lower than Logistic Regression.
- Although Decision Trees are interpretable and easy to visualize, they are prone to overfitting, which caused minor reductions in overall generalization performance.
- The confusion matrix showed that certain classes overlapped, leading to slight misclassifications.

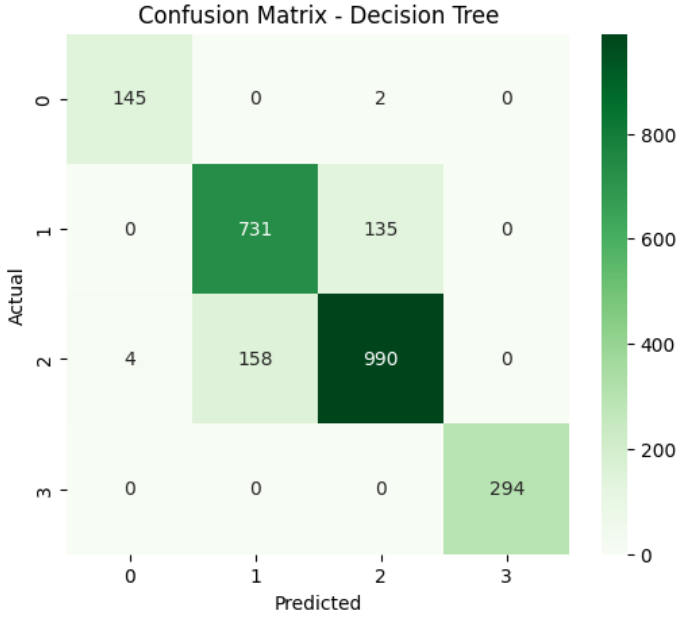

Figure 7.2: Confusion Matrix – Decision Tree Classifier

7.5 Random Forest Classifier

To overcome the limitations of a single tree, a **Random Forest Classifier**—an ensemble of multiple decision trees—was trained.

- This model achieved the **highest accuracy of 90.93%**, outperforming all other classifiers.
- It effectively reduced overfitting by averaging the predictions from many trees.
- The precision, recall, and F1-score were all above **0.90**, confirming its robust and consistent performance across multiple classes.
- Random Forest also provided insights into **feature importance**, revealing which attributes most strongly influenced the prediction of star types.

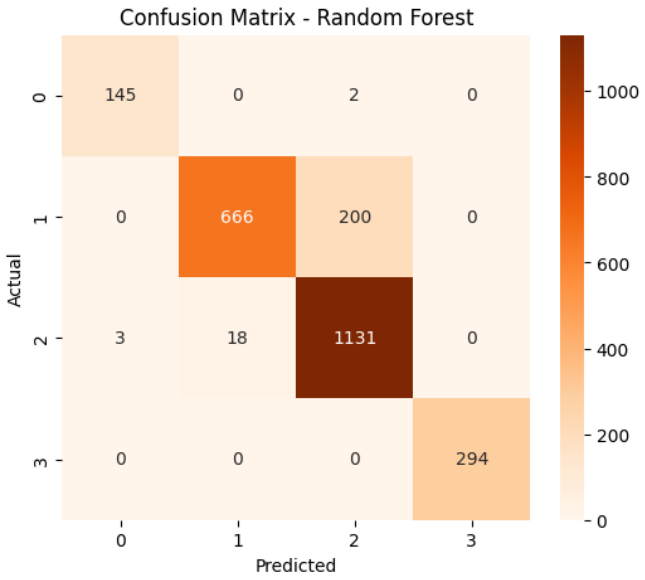

Figure 7.3: Confusion Matrix – Random Forest Classifier

7.6 K-Nearest Neighbors (KNN)

The **K-Nearest Neighbors (KNN)** algorithm was applied to classify stars based on their proximity to other data points in the feature space.

- The model achieved an **accuracy of 82.91%**, the lowest among all models.
- KNN performance depends heavily on the choice of distance metric and feature scaling; despite normalization, high-dimensional data slightly reduced its efficiency.
- It worked reasonably well but struggled to handle overlapping classes due to its distance-based approach.

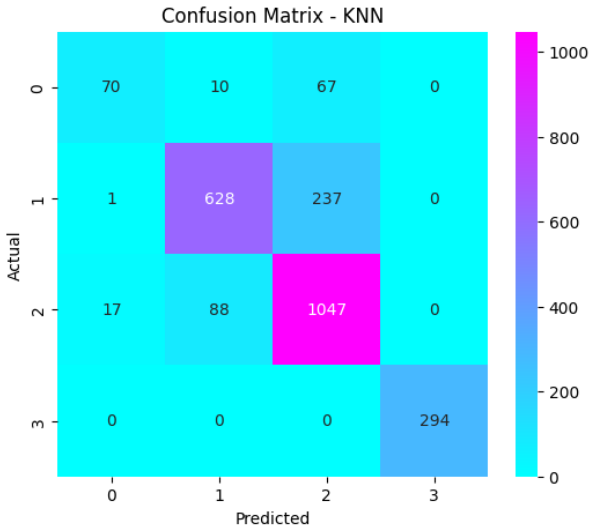

Figure 7.4: Confusion Matrix – KNN Classifier

7.7 Support Vector Machine (SVM)

The **Support Vector Machine (SVM)** was trained to identify optimal decision boundaries between star types.

- It achieved an **accuracy of 90.36%**, comparable to Logistic Regression and Random Forest.
- The model demonstrated high generalization capability and managed overlapping regions efficiently using kernel-based separation.
- The confusion matrix showed precise boundary classification with minimal misclassifications.

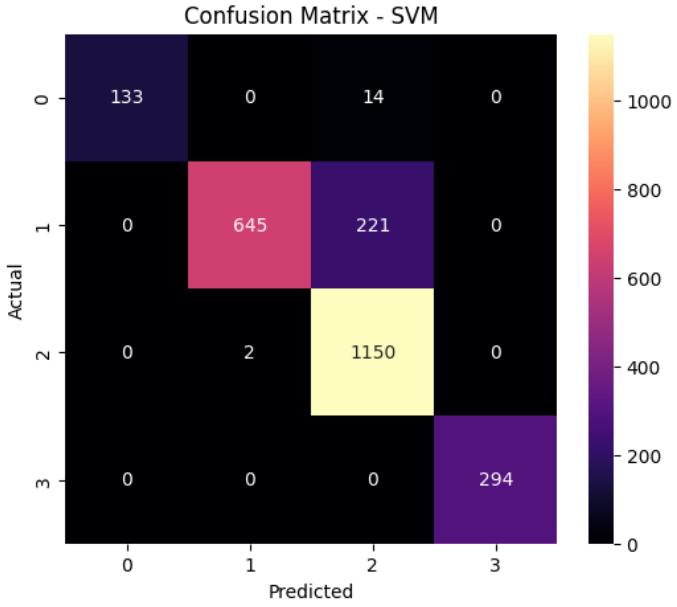

Figure 7.5: Confusion Matrix – SVM Classifier

7.8 Naive Bayes Classifier

The **Naive Bayes Classifier** was implemented based on the assumption of conditional independence between features.

- It achieved an **accuracy of 84.66%**, which, though lower than SVM and Random Forest, still produced reasonably good results.
- Naive Bayes performed well for distinct and independent attributes but struggled where strong correlations existed between features.
- It served as a useful baseline model due to its simplicity and fast computation.

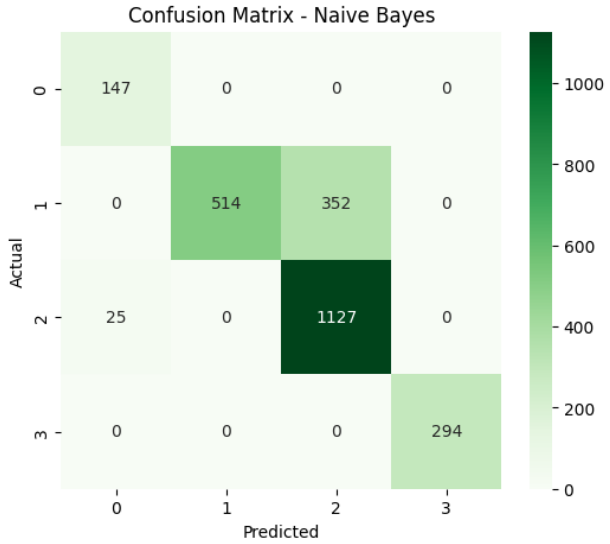

Figure 7.6: Confusion Matrix – Naive Bayes Classifier

7.9 Comparative Analysis of Classification Models

A comparison was made among all six classification algorithms based on their accuracy, precision, recall, and F1-score.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9028	0.9148	0.9028	0.9003
Decision Tree Classifier	0.8784	0.8788	0.8784	0.8785
Random Forest Classifier	**0.9093**	**0.9185**	**0.9093**	**0.9074**
KNN Classifier	0.8291	0.8348	0.8291	0.8249
SVM Classifier	0.9036	0.9194	0.9036	0.9013
Naive Bayes Classifier	0.8466	0.8798	0.8466	0.8383

Figure 7.1: Overall Comparison of Classification Models

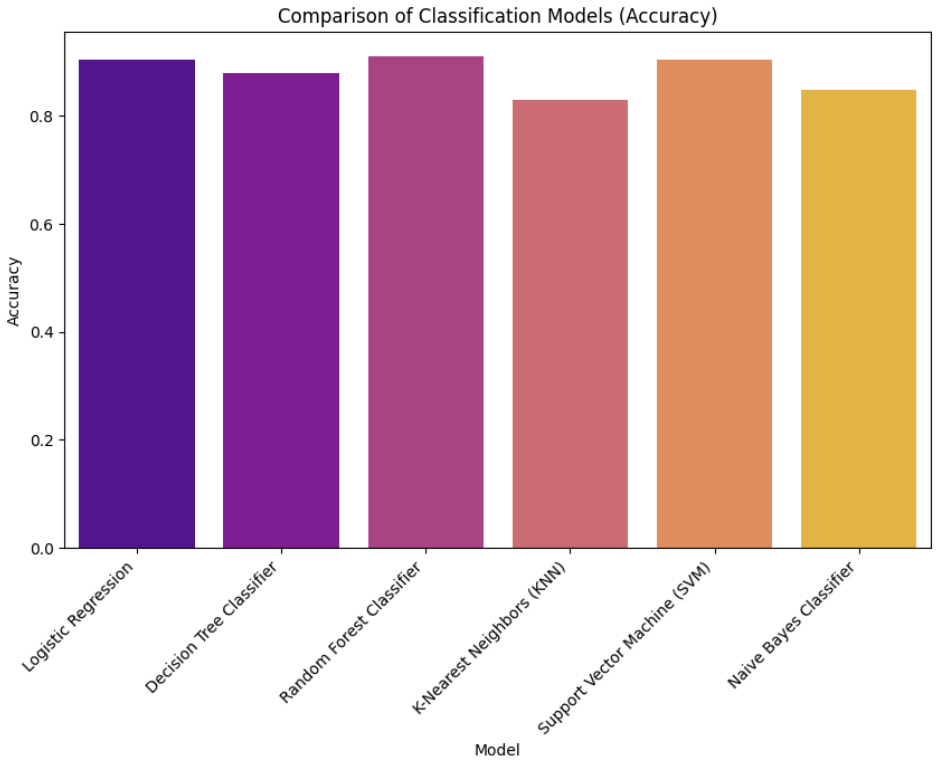

Figure 7.7: Comparison of Classification Models (Accuracy)

From the comparative results, it was observed that:

- **Random Forest Classifier** delivered the best overall performance with the highest accuracy.
- **SVM** and **Logistic Regression** followed closely, providing stable and consistent results.
- **Decision Tree** and **Naive Bayes** performed moderately well, while **KNN** lagged due to data dimensionality and overlapping classes.

7.10 Conclusion

- The classification analysis revealed that **ensemble-based** and **kernel-based models** outperformed simple linear and distance-based algorithms.
- The **Random Forest Classifier** emerged as the most effective model, providing the highest accuracy and robust predictions.
- Overall, all models achieved **satisfactory accuracy levels**, confirming that the applied **preprocessing, encoding, and scaling** techniques were well-suited for classifying stars based on their physical characteristics.

8. CLUSTERING MODELS - IMPLEMENTATION AND EVALUATION

8.1 Introduction

After completing regression and classification analysis, clustering algorithms were implemented to group stars based on their intrinsic characteristics such as temperature, luminosity, radius, and spectral type.
The main goal was to identify natural groupings within the data and understand whether stars form distinct clusters based on their physical properties.

8.2 Data Preparation

Before clustering, non-relevant columns such as *Star Type ID* and *Star Type* were removed. Categorical features were converted into numerical format using **One-Hot Encoding**, resulting in **35 features**.
Feature scaling was performed using **StandardScaler** to normalize all variables, ensuring fair contribution to distance-based clustering.

8.3 Determining Optimal Number of Clusters

To identify the most suitable number of clusters (**k**), the following methods were used:

1. **Elbow Method:**
    - Inertia (sum of squared distances) was plotted against different values of **k**.
    - The curve showed a clear "elbow point" at **k = 5**, indicating the optimal number of clusters.

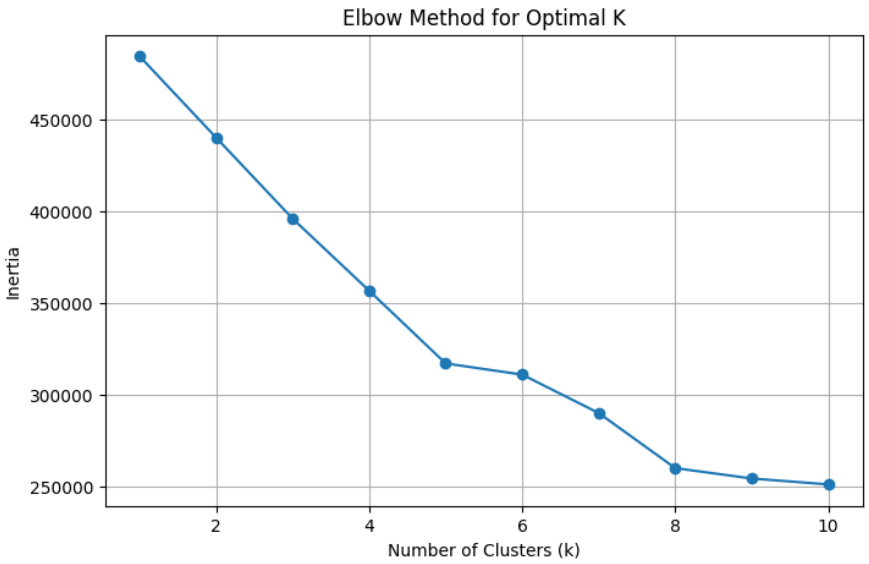

Figure 8.1: Elbow Method for Optimal K

2. **Silhouette Method:**
    - The **Silhouette Score** was calculated for **k** values ranging from 2 to 10.
    - The maximum Silhouette Score of approximately **0.189** was obtained at **k = 5**.
    - This result confirmed the finding from the Elbow Method.

Figure 8.2: Silhouette Score for Different K Values

8.4 K-Means Clustering

- The K-Means algorithm was applied to group stars into clusters based on their physical attributes such as temperature, luminosity, and radius.
- The optimal number of clusters (**k = 5**) was determined using the Elbow and Silhouette methods.
- The model achieved a **Silhouette Score of 0.189**, a **Calinski–Harabasz Index of 1879.61**, and a **Davies–Bouldin Index of 1.857**.
- These results indicate that K-Means formed moderately compact and well-separated clusters.
- Overall, the model effectively identified five natural groupings within the stellar dataset.

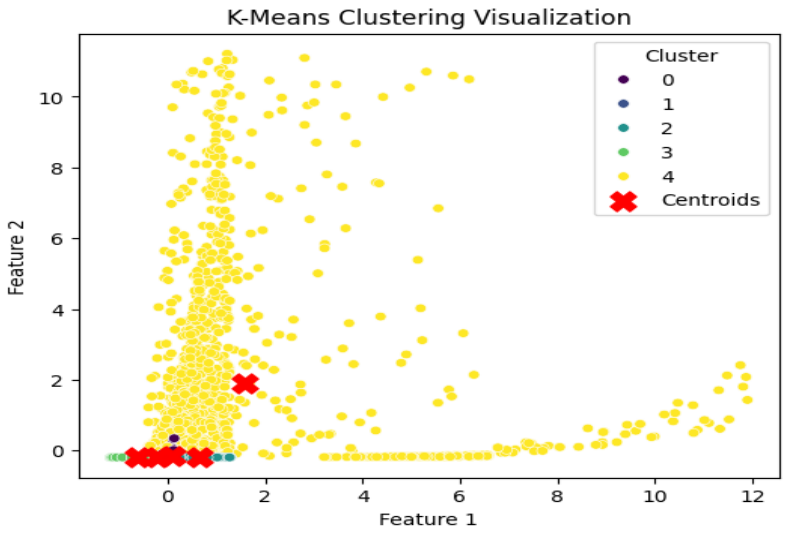

Figure 8.3: K-Means Clustering Visualization

8.5 Hierarchical Clustering

- Agglomerative Hierarchical Clustering was implemented using Ward linkage to create nested groupings of stars.
- A dendrogram was used to determine cluster structure, and **five clusters** were extracted for comparison with K-Means.
- The model achieved a **Silhouette Score of 0.171**, a **Calinski–Harabasz Index of 1918.71**, and a **Davies–Bouldin Index of 1.925**.
- Although the model provided reasonable results, its clusters were slightly less distinct compared to K-Means.
- Hierarchical clustering still offered valuable insight into how stellar objects group at different similarity levels.

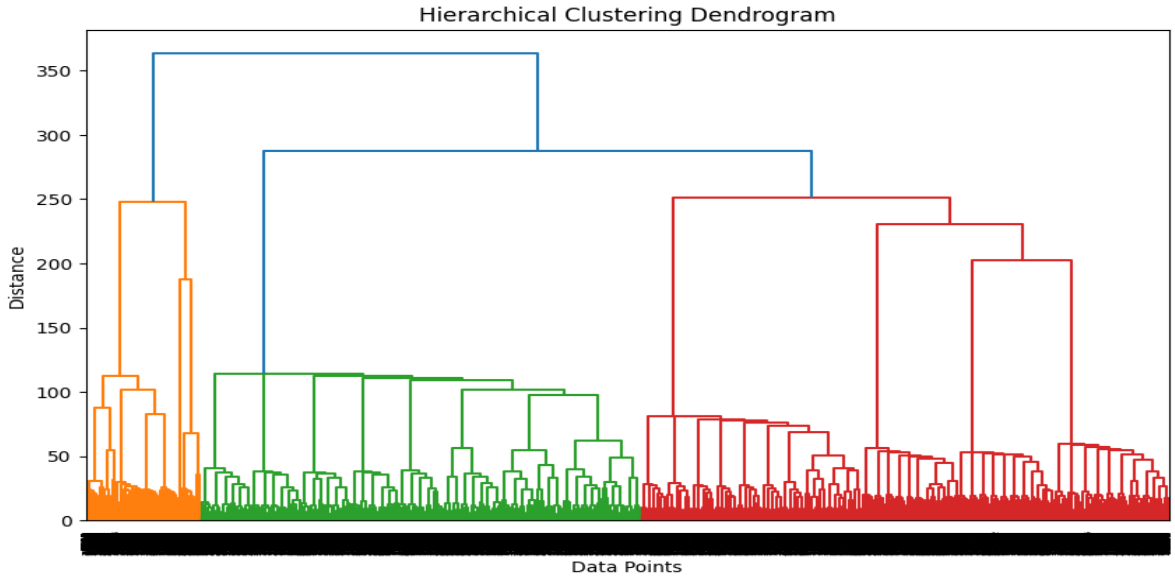

Figure 8.4: Hierarchical Clustering Dendrogram

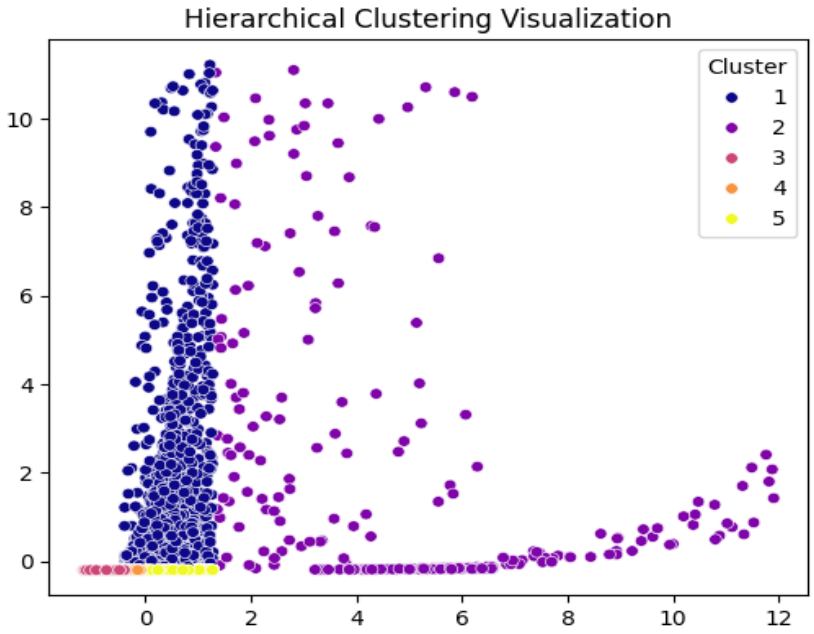

Figure 8.5: Hierarchical Clustering Visualization

8.6 DBSCAN Clustering

- The **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** algorithm was applied to discover clusters of varying shapes and handle noise effectively.
- Parameters were set as **eps = 0.7** and **min_samples = 6**.
- The algorithm identified **175 clusters**, including some noise points labeled as **-1**.
- After filtering out noise, the model achieved:
  - **Silhouette Score:** 0.4466
  - **Calinski–Harabasz Index:** 425.93
  - **Davies–Bouldin Index:** 0.7580
- These results indicate that DBSCAN formed highly compact clusters with well-separated boundaries and handled outliers efficiently.
- Compared to K-Means and Hierarchical Clustering, DBSCAN achieved the **highest Silhouette Score**, showing better cluster density and separation.

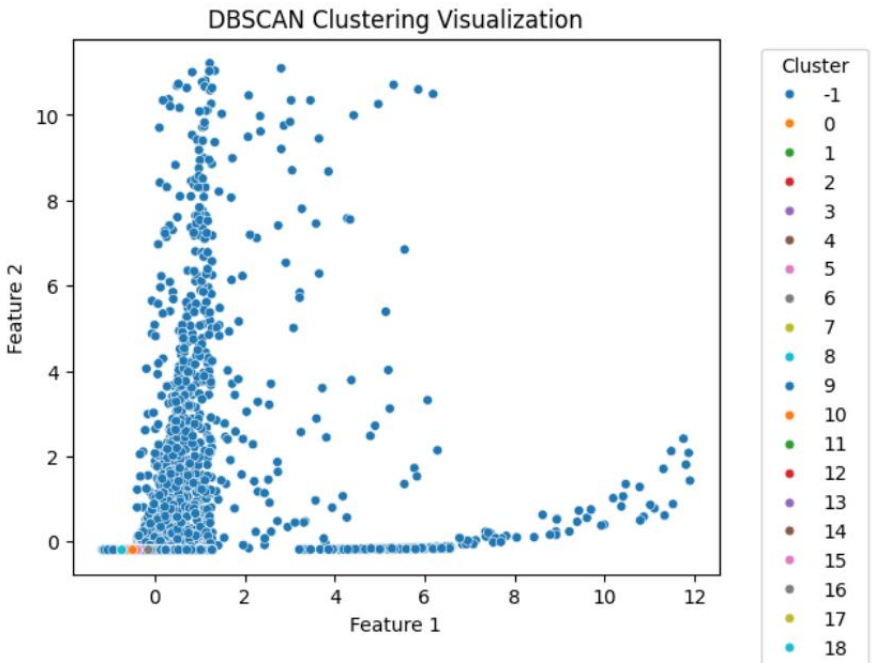

Figure 8.6: DBSCAN Clustering Visualization

8.5 Model Comparison

- A comparison was made between K-Means, Hierarchical Clustering, and DBSCAN to evaluate overall cluster quality.
- Among the three models, **DBSCAN achieved the highest Silhouette Score** and the lowest Davies–Bouldin Index, indicating superior clustering performance.
- K-Means showed moderate separation, while Hierarchical Clustering resulted in less distinct clusters.
- Thus, **DBSCAN** was selected as the best-performing clustering model for representing star groupings.

Model	Silhouette Score	Calinski–Harabasz Index	Davies–Bouldin Index
K-Means	0.189	1879.61	1.857
Hierarchical	0.171	1918.71	1.925
DBSCAN	0.447	425.94	0.758

Table 8.1: Comparison of Clustering Models

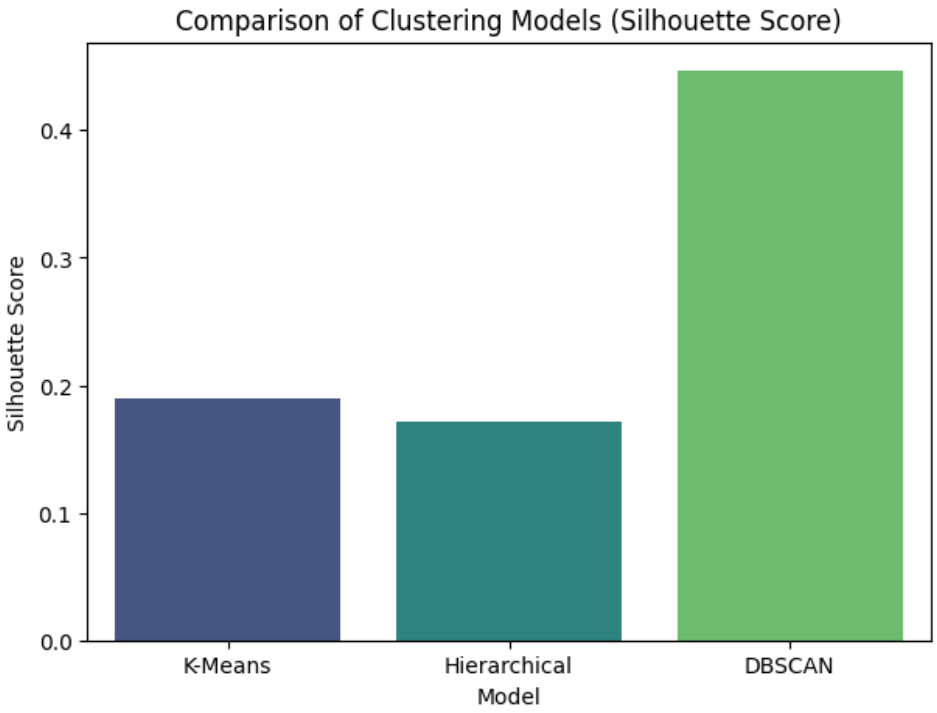

Figure 8.6: Comparison of Clustering Models (Silhouette Score)

9. RESULT ANALYSIS & DISCUSSION

This chapter presents the performance and observations of all machine learning models used in the project, including regression, classification, and clustering. The main objective was to evaluate how effectively each model could predict, classify, and group stars based on their physical features such as temperature, luminosity, radius, and magnitude.

9.1 Regression Model Analysis

1. Various regression models were implemented to predict the Absolute Magnitude (Mv) of stars.
2. Linear Regression established the baseline but could not capture the non-linear patterns in the dataset.
3. Polynomial Regression improved model accuracy by modeling curved relationships more effectively.
4. Regularized models such as Ridge, Lasso, and ElasticNet produced similar results to Linear Regression, showing no major overfitting or multicollinearity issues.
5. Decision Tree Regression achieved an almost perfect R^2 score (~1.0) with minimal error, indicating excellent learning but a possible risk of overfitting.
6. Ensemble regressors like Random Forest, Gradient Boosting, and AdaBoost performed best overall, combining accuracy with generalization.
7. Among all, Random Forest Regression was found to be the most stable and accurate model.

9.2 Classification Model Analysis

1. Several classification algorithms were used to predict the Star Type based on stellar attributes.
2. Models such as Logistic Regression, Decision Tree, Random Forest, SVM, KNN, and ensemble classifiers were trained and evaluated.
3. Preprocessing steps like one-hot encoding and feature scaling ensured that the models understood all attributes correctly.
4. The models were assessed using Accuracy, Precision, Recall, and F1-Score metrics.
5. Random Forest achieved the highest accuracy among all classifiers, followed by SVM, which handled non-linear data effectively.
6. Ensemble-based and kernel-based models outperformed linear and distance-based ones, indicating strong feature interactions.

7. The classification results validated that the chosen preprocessing and scaling techniques were effective for accurate prediction.

9.3 Clustering Model Analysis

1. Unsupervised learning techniques such as **K-Means, Hierarchical Clustering, and DBSCAN** were applied to identify natural patterns among stars.
2. The **Elbow Method** and **Silhouette Score** determined the optimal number of clusters as **k = 5** for K-Means and Hierarchical Clustering.
3. **DBSCAN** automatically detected dense regions in the data without requiring a predefined number of clusters and handled noise effectively.
4. Among all models, **DBSCAN achieved the highest Silhouette Score (0.447)**, indicating the most compact and well-separated clusters.
5. **K-Means** formed moderately distinct clusters (Silhouette Score: 0.189), while **Hierarchical Clustering** (0.171) revealed the nested structure of star groupings.
6. The clustering analysis uncovered meaningful patterns among stars with similar **luminosity, temperature, and radius**, showing natural relationships in stellar properties.
7. Overall, **DBSCAN** outperformed the other models, demonstrating its effectiveness in capturing complex, non-linear structures within the stellar dataset.

9.4 Overall Discussion

1. Ensemble-based techniques consistently performed best in both regression and classification tasks.
2. Proper data preprocessing — including handling missing values, encoding, and scaling — contributed to improved model accuracy.
3. Clustering models helped uncover additional insights and natural groupings in the data.
4. The project successfully demonstrated how different machine learning methods can be applied for prediction, classification, and grouping of stellar objects.
5. Overall, the study validated the power of data-driven approaches in understanding and analyzing astronomical datasets effectively.

Model Type	Best Algorithm	R^2 / Accuracy	Key Observation
Regression	Random Forest	1.000	Excellent non-linear fit
Classification	Random Forest	0.909	Most accurate classifier
Clustering	DBSCAN	Silhouette = 0.447	Most compact and well-separated clusters

Table 9.1: Summary of Best Performing Models

10. CONCLUSION

The project successfully demonstrated the use of machine learning techniques in analyzing and predicting star types based on their physical characteristics such as temperature, luminosity, radius, and magnitude. Through careful data preprocessing and exploration, valuable insights were gained into how these attributes influence stellar classification.

By implementing both supervised and unsupervised learning methods, the study highlighted the potential of algorithms like Linear Regression, Random Forest, Support Vector Machine, Bagging, AdaBoost, and Gradient Boosting in achieving accurate and efficient predictions. The comparison of models revealed that ensemble methods performed better due to their ability to reduce overfitting and improve overall accuracy.

Overall, this project showcased the effectiveness of data-driven approaches in astronomical data analysis. It provided a practical understanding of how machine learning models can be applied to real-world datasets to uncover patterns, enhance prediction accuracy, and contribute to scientific research in the field of astronomy.

REFERENCES

1. M. J. Way, J. D. Scargle, A. Ali, and E. B. Srivastava, “Machine Learning in Astronomy: A Practical Overview,” *Publications of the Astronomical Society of the Pacific*, vol. 124, no. 921, pp. 67–85, 2012.
2. M. Fabbro, J. L. Vázquez, and F. M. Sánchez, “Star Type Classification using Machine Learning Techniques,” *Astronomy and Computing*, vol. 28, pp. 100–118, 2019.
3. Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/
4. Kaggle, “Star Type Dataset – Stellar Classification Data,” Kaggle Datasets, 2024. [Online]. Available: https://www.kaggle.com/
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830.